

RICORSIONE E ITERAZIONE

LA RICORSIONE È UNO STRUMENTO MATEMATICO MOLTO ELEGANTE, MA NON ESISTE NATIVAMENTE NEI CALCOLATORI. OGNI VOLTA CHE VIENE EFFETTUATA UNA CHIAMATA RICORSIVA, IL SISTEMA OPERATIVO DEVE SIMULARE QUESTO COMPORTAMENTO ALLOCANDO UN RECORD DI ATTIVAZIONE IN UNA STACK DI MEMORIA, CON IL RISCHIO DI ANDARE IN $\Rightarrow$ STACK OVERFLOW.

L'OBIETTIVO DELLA TRADUZIONE DI UN ALGORITMO DA RICORSIVO A ITERATIVO È L'ELIMINAZIONE DI QUESTA DIPENDENZA DAL SISTEMA, FISSANDO IL FLUSSO MANUALMENTE, TRAMITE UN APPROCCIO ITERATIVO CONTROLLATO.

ARCHITETTURA

PER TRADURRE UN ALGORITMO RICORSIVO, L'ARCHITETTURA SI BASA SU UN GRANDE CICLO WHILE, GOVERNATO DA 3 COMPONENTI FONDAMENTALI:

1. MEMORIA DI LAVORO: SOSTITUISCE IL RECORD DI ATTIVAZIONE. HA IL COMPITO DI SALVARE ESCLUSIVAMENTE IL CONTESTO CHE SARÀ NECESSARIO AL RITORNO DELLA CHIAMATA
2. LA VARIABILE `start`: È IL SEMAFORO DIREZIONALE DELL'ALGORITMO:
 - `start = TRUE`: FASE DI DISCESA (INIZIALIZZAZIONE E AVVIO DELLA CHIAMATA)
 - `start = FALSE`: FASE DI RISALITA (RECUPERO DATI DALLA STACK, RIPRESA ESECUZIONE)
3. LA VARIABILE `last`: NECESSARIA PER ORIENTARSI SE CI SONO PIÙ CHIAMATE RICORSIVE NELLO STESSO ALGORITMO. REGISTRA L'IDENTIFICATIVO DELL'ULTIMA SOTTOCHIAMATA APPENA CONCLUSA PER FAR CAPIRE AL CICLO DA QUALE RAMO STIAMO TORNANDO E DA QUALE ISTRUZIONE RIPARTIRE

ECEZIONE - RICORSIONE IN CODA

SE LA CHIAMATA RICORSIVA È RIGOROSAMENTE L'ULTIMA ISTRUZIONE DELL'ALGORITMO, LA STRUTTURA DECADE. VERRÀ SOLO EFFETTUATO UN AGGIORNAMENTO DEI PARAMETRI IN UN CICLO STANDARD

ESEMPIO 1

FUNZIONE FATTORIALE

IL CALCOLO DEL FATTORIALE È DEFINITO, PER INDUZIONE, COME

$$n! = n \cdot (n-1)!$$

CON CASO BASE $0! = 1$

VERSIONE RICORSIVA

FATTORIALE (n)

IF $n \geq 0$

RETURN 1

ELSE

RETURN $n \cdot \text{FATTORIALE}(n-1)$

POICHÉ LA FUNZIONE RICORSIVA EFFETTUA UNA SOLA SOTTOCHIAMATA, LA TRADUZIONE ITERATIVA NON HA BISOGNO DELLA VARIABILE LAST (C'È UN UNICO PUNTO DI RITORNO). LO STACK SERVE UNICAMENTE A SALVARE IL NUMERO CORRENTE PRIMA DELLA DISCESA, COSÌ CHE POSSA ESSERE MOLTIPLICATO AL RITORNO.

VERSIONE ITERATIVA

FATTORIALE-ITC(m)

CURRENT NUMBER = m

CALLSTACK = NIL

ISDESCENDING = TRUE

WHILE (ISDESCENDING == TRUE OR CALLSTACK != NIL)

IF ISDESCENDING == TRUE

IF CURRENT NUMBER = 0

RESULT = 1

ISDESCENDING = FALSE

ELSE

CALLSTACK = PUSH (CALLSTACK, CURRENT NUMBER)

CURRENT NUMBER = CURRENT NUMBER - 1

ELSE

SUSPENDED NUMBER = POP (CALLSTACK)

RESULT = SUSPENDED NUMBER * RESULT

RETURN RESULT

ESEMPIO 2

MERGE SORT

IL MERGE SORT È UN ALGORITMO BASATO SUL DIVIDE ET IMPERA, EFFETTUA DUE CHIAMATE RICORSIVE CONSECUTIVE SULLO STESSO ARRAY, POI L'OPERAZIONE DI MERGE. QUESTO ELEVA LA COMPLESSITÀ RISPETTO AL FATTORIALE

VERSIONE RICORSIVA

MERGE SORT (A, start, end)

IF START < END

mid = (start + end) / 2

MERGE SORT (A, start, mid)

MERGE SORT (A, mid + 1, end)

MERGE (A, start, mid, end)

A DIFFERENZA DEL FATTORIALE IN CUI USAVAMO SUBITO POP, QUI IL CONTESTO SALVATO NELLO STACK DEVE SOPRAVVIVERE ALLA PRIMA CHIAMATA PER ESSERE USATO NELLA SECONDA. PERTANTO, NELLA FASE DI RISALITA USEREMO UN'OPERAZIONE DI PEER, PER FARE POP SOLO PRIMA DEL MERGE

CURRSTART = START

CURREND = END

ISDESCENDING = TRUE

STARTSTACK = NIL

MIDSTACK = NIL

ENDSTACK = NIL

WHILE (ISDESCENDING == TRUE || ENDSTACK != NIL)

IF ISDESCENDING == TRUE

IF CURRSTART < CURREND // PASSO RICORSIVO

CURRMID = (CURRSTART + CURREND) / 2

STARTSTACK = PUSH(STARTSTACK, CURRSTART)

MIDSTACK = PUSH(MIDSTACK, CURRMID)

ENDSTACK = PUSH(ENDSTACK, CURREND)

CURREND = CURRMID

ELSE // CASO BASE: ARRAY DI 1 ELEMENTO

LASTCOMPLETED END = CURREND

ISDESCENDING = FALSE

ELSE //

CURRSTART = PEEK(STARTSTACK)

CURRMID = PEEK(MIDSTACK)

CURREND = PEEK(ENDSTACK)

IF LASTCOMPLETED END == CURRMID

CURRSTART = CURRMID + 1

ISDESCENDING = TRUE

ELSE

POP(STARTSTACK)

POP(MIDSTACK)

POP(ENDSTACK)

MERGE(A, CURRSTART, CURRMID, CURREND)

LASTCOMPLETED END = CURREND

ESEMPIO MERGESORT ITERATIVO

$A = [38, 27]$

Stato INIZIALE

38	27
----	----

0 1

CURRSTART = 0

CURRMID = NIL

CURREND = 1

ISDESC. = TRUE

LAST = -1

START STACK

☐

MIDSTACK

☐

ENDSTACK

☐

GIRO 1

ISDESC == TRUE E CURRSTART < CURREND QUINDI CALCOLO CURRMID = 0,

SALVO STATO ATTUALE NELLO STACK:

START STACK

CURRSTART = 0

MIDSTACK

CURRMID = 0

ENDSTACK

CURREND = 1

IN FINE L'IPOTESI $CURREND = CURRMID \Rightarrow CURREND = 0$

CURRSTART = 0

CURRMID = 0

CURREND = 0

ISDESC = TRUE

LAST = -1

START STACK

CURRSTART = 0

MIDSTACK

CURRMID = 0

ENDSTACK

CURREND = 1

GIRO 2

ISDESCENDING == TRUE MA $(CURRSTART < CURREND) == FALSE$,

QUINDI ENTRO NELL'ELSE, SETTO LAST = 0 (ARRAY DI UN

SOLO ELEMENTO, È GIÀ ORDINATO, QUINDI IL "LATO" SX È A POSTO)

CURRSTART = 0

CURRMID = 0

CURREND = 0

ISDESC = TRUE

LAST

START STACK

CURRSTART = 0

MIDSTACK

CURRMID = 0

ENDSTACK

CURREND = 1

Giro 3

ISDESCENDING != TRUE, FACCIAMO PEEK SUGLI STACK E NOTIAMO $LAST == CURRMID$, ABBIAMO APPENA FINITO DI ORDINARE IL LATO SX, QUINDI DOBBIAMO INIZIARE COL DESTRO: IMPOSTIAMO $CURRSTART = CURRMID + 1 = 1$ E $CURREND = 1$ (MAX INDICE ARRAY), REIMPOSTIAMO $ISDESC = TRUE$

$CURRSTART = 1$

START STACK

$CURRSTART = 0$

MIDSTACK

$CURRMID = 0$

$CURRMID = 0$

$CURREND = 1$

END STACK

$CURREND = 1$

$ISDESC = TRUE$

$LAST = 0$

Giro 4

$ISDESC == TRUE$, MA $(CURRSTART < CURREND) == FALSE$, QUINDI CI TROVIAMO DI NUOVA CON UN ARRAY ORDINATO DA 1 ELEMENTO, SETTIAMO $LAST = 1$ E $ISDESC = FALSE$

$CURRSTART = 1$

START STACK

$CURRSTART = 0$

MIDSTACK

$CURRMID = 0$

$CURRMID = 0$

$CURREND = 1$

END STACK

$CURREND = 1$

$ISDESC = FALSE$

$LAST = 1$

Giro 5

$ISDESC != TRUE$, EFFETTUIAMO IL PEEK, $CURRSTART = 0$, $CURRMID = 0$, $CURREND = 1$, $LAST != CURRMID$, QUINDI SIA SX CHE DX SONO ORDINATI, SVUOTIAMO LO STACK. CHIAMIAMO $MERGE(A, 0, 0, 1)$, 27 E 28 SI INVERTONO E $LAST = 1$

$CURRSTART = 0$

START STACK

MIDSTACK

$CURRMID = 0$

$CURREND = 1$

END STACK

$ISDESC = FALSE$

$LAST = 1$

FINE: $ISDESC = FALSE$ E $STACK = NIL \Rightarrow A$ È ORDINATO. $A = [27, 38]$

ESEMPIO 3

FIBONACCI

L'ALGORITMO RICORRENTE PER CALCOLARE L' n -ESIMO NUMERO DELLA SEQUENZA DI FIBONACCI È DEFINITO COME
 $FIB(n) = FIB(n-1) + FIB(n-2)$, $FIB(0) = 0$, $FIB(1) = 1$.
VISTE LE 2 CHIAMATE RICORSIVE CONSECUTIVE, AVREMO NUOVAMENTE BISOGNO DELLA VARIABILE `last` PER CAPIRE DA QUALE DELLE DUE CHIAMATE STIAMO TORNANDO. C'È PERÒ UNA DIFFERENZA FONDAMENTALE COL MERGE SORT, IL VALORE DI RITORNO:

- IL MERGE SORT LAVORA IN PLACE SU UN ARRAY FISSO IN MEMORIA
- FIBONACCI RESTITUISCE UN VALORE NUMERICO CHE DEVE ESSERE CATTURATO E SOMMATO

QUESTO IMPLICA LA NECESSITÀ DI MEMORIA AGGIUNTIVA PER SALVARE IL VALORE DELLA PRIMA CHIAMATA, EFFETTUARE LA SECONDA E, SOLO ALLA FINE, POTREMMO ESTRARRE IL PRIMO VALORE, SOMMARLO AL SECONDO E PASSARE AL LIVELLO SUPERIORE.

VERSIONE RICORSIVA

$FIB-R(n)$

IF $n \leq 1$

RETURN n

ELSE

RETURN $FIB(n-1) + FIB(n-2)$

VERSIONE ITERATIVA

FIB-ITC(n)

CURR N = n

LAST COMPLETED N = -1

IS DESCENDING = TRUE

RESULT = -1

STACK N = NIL

STACK PARTIAL RESULT = NIL

WHILE (IS DESCENDING == TRUE || STACK N $\neq$ NIL)

IF (IS DESCENDING == TRUE)

IF (CURR N $\leq$ 1)

RESULT = CURR N

IS DESCENDING = FALSE

LAST COMPLETED N = CURR N

ELSE

STACK N = PUSH (STACK N, CURR N)

CURR N = CURR N - 1

ELSE

SUSPENDED N = PEEK (STACK N)

IF (LAST COMPLETED N == SUSPENDED N - 1)

STACK PARTIAL RESULT = PUSH (STACK PARTIAL RESULT, RESULT)

CURR N = SUSPENDED N - 2

IS DESCENDING = TRUE

ELSE

EXTRACTED RESULT = POP (STACK PARTIAL RESULT)

RESULT = RESULT + EXTRACTED RESULT

POP (STACK N)

LAST COMPLETED N = SUSPENDED N

RETURN RESULT

RICORSIONE NEGLI ALBERI BINARI

SE FINORA, PER LA TRADUZIONE DA RICORSIVO A ITERATIVO, LA NAVIGAZIONE AVVENIVA TRAMITE INDICI O VALORI, NEGLI ABR VEDIAMO UN LEGGERO CAMBIO DI PARADIGMA, LA NAVIGAZIONE DEI NODI AVVIENE TRAMITE PUNTATORI (`left`, `right`).

NEGLI ALGORITMI PRECEDENTI, LA VARIABILE `LAST` DISCRIMINAVA IL PUNTO DI RITORNO CONFRONTANDO VALORI NUMERICI UNIVOCI, NEGLI ABR QUESTO MECCANISMO SI ROMPE NON APPENA RAGGIUNGIAMO UNA FOGLIA, PERCHÉ ENTRAMBI I SUOI PUNTATORI `left` E `right` SARANNO `NIL`, NON PERMETTENDO LA DISTINZIONE TRA IL RITORNO DAL FIGLIO DESTRO O DAL SINISTRO.

PER RISOLVERE QUESTO PROBLEMA, LA VARIABILE `LAST` IMMAGAZZERÀ IL PUNTATORE DELL'ULTIMO NODO ESPORATO, L'IMPLICAZIONE È CHE IL BIVIO LOGICO SARÀ BASATO SULL'UGUAGLIANZA TRA PUNTATORI E NON TRA NUMERI. AVREMO QUINDI LA CONDIZIONE

`IF (LAST == CURRENTNODE->LEFT AND CURRENTNODE != NIL)`

ESEMPIO PRINT-TREE (VISITA POST ORDER)

PRINT-TREE-REC (NODE)

`IF NODE != NIL`

`PRINT-TREE-REC (NODE->LEFT)`

`PRINT-TREE-REC (NODE->RIGHT)`

`PRINT-TREE-REC (NODE->KEY)`

VEDIAMO ORA LA TRADUZIONE ITERATIVA

PRINT-TREE-IT (ROOT)

CURRNODE = ROOT

ISDESC = TRUE

LASTVISITED = NIL

NODESTACK = NIL

WHILE (ISDESC == TRUE || STACKNODE != NIL)

IF (ISDESCENDING == TRUE) // FASE DI DISCESA

IF (CURRNODE != NIL)

NODESTACK = PUSH(NODESTACK, CURRNODE) // SALVIAMO IL

CURRNODE = CURRNODE → LEFT

NODO PADRE

ELSE // SE CURRNODE == NIL ABBIAMO SUPERATO LO FOLIO

LASTVISITED = CURRNODE

ISDESC = FALSE // RISALIAMO

ELSE

SUSPENDEDNODE = PEEK(NODESTACK)

IF (LASTVISITED == SUSPENDEDNODE → LEFT && SUSPENDEDNODE → RIGHT != NIL)

CURRNODE = SUSPENDEDNODE → RIGHT // VISITIAMO L'ALZABRODX

ISDESCENDING = TRUE

ELSE

PRINT (SUSPENDEDNODE → KEY)

POP (STACK NODE)

LASTVISITED NODE = SUSPENDED NODE

ESEMPIO HEIGHT

HEIGHT-REC(NODE)

HEIGHT = -1

IF NODE != NIL

LEFT HEIGHT = HEIGHT-REC(NODE → LEFT)

RIGHT HEIGHT = HEIGHT-REC(NODE → RIGHT)

HEIGHT = 1 + MAX(LEFT HEIGHT, RIGHT HEIGHT)

RETURN HEIGHT

HEIGHT-IT (ROOT)

CURRNODE = ROOT

ISDESC = TRUE

LASTVISITED = NIL

CURRHEIGHT = -1

NODESTACK = NIL

HEIGHTSTACK = NIL

WHILE (ISDESC || NODESTACK != NIL)

IF (ISDESC)

IF (CURRNODE != NIL)

NODESTACK = PUSH (NODESTACK, CURRNODE)

CURRNODE = CURRNODE → LEFT

ELSE

CURRHEIGHT = -1

LASTVISITED = CURRNODE

ISDESC = FALSE

ELSE

SUSPENDEDNODE = PEEK (NODESTACK)

IF (LASTVISITED == SUSPENDEDNODE → LEFT)

HEIGHTSTACK = PUSH (HEIGHTSTACK, CURRHEIGHT)

CURRNODE = SUSPENDEDNODE → RIGHT

ISDESC = TRUE

ELSE

LEFTHEIGHT = POP (HEIGHTSTACK)

RIGHTHEIGHT = CURRHEIGHT

CURRHEIGHT = 1 + MAX (LEFTHEIGHT, RIGHTHEIGHT)

POP (NODESTACK)

LASTVISITED (SUSPENDEDNODE)

RETURN CURRHEIGHT

IL PATTERN UNIVERSALE

LA TRADUZIONE DA RICORSIVO A ITERATIVO, CHE RICORDIAMO, È L'OPERAZIONE SIMULATA DALL'AMBIENTE DI ESECUZIONE AD OGNI CHIAMATA A FUNZIONE. ESSENDO A TUTTI GLI EFFETTI UNA PROCEDURA MECCANICA, È DETERMINISTICA, NON RICHIEDE QUINDI INTUIZIONE CREATIVA, MA L'APPLICAZIONE DI UN PATTERN RIGIDO, CHE POSSIAMO RIASSUMERE IN 4 STEP:

1. ANALISI DELLA MEMORIA
2. IDENTIFICAZIONE DELLA "BUSSOLA"
3. GESTIONE DEL CASO BASE
4. MAPPATURA DELLE AREE DI ESECUZIONE

VEDIAMOLE NEL DETTAGLIO

1. ANALISI DELLA MEMORIA (QUANTI STACK SERVONO)

PRIMA DI SCRIVERE IL CICLO WHILE, GUARDIAMO LA FIRMA DELLA FUNZIONE RICORSIVA, AVREMO BISOGNO DI:

- STACK DEL CONTESTO (OBBLIGATORIO): NECESSARIO A SALVARE I PARAMETRI ATTUALI PRIMA DI "SCENDERE" (E.G. NODESTACK, PARAMSTACK)
- STACK DEI RISULTATI (OPZIONALE): SE LA FUNZIONE RESTITUISCE UN VALORE CHE SARÀ COMBINATO CON LE CHIAMATE SUCCESSIVE, SERVE UNO STACK AUSILIARIO, SE LA FUNZIONE NON RESTITUISCE NULLA, QUESTO STACK NON SERVE.

2. IDENTIFICAZIONE DELLA 'BUSSOLA' (COME CI ORIENTIAMO)

DOBBIAMO DEFINIRE IL TIPO DELLA VARIABILE LAST E LA CONDIZIONE PER LANCIARE LA SECONDA CHIAMATA

- SE NAVIGHIAMO UN ALBERO LA BUSSOLA SALVA UN PUNTATORE ALL'ULTIMO NODO VISITATO, CONTROLLANDO SE STIAMO TORNANDO DA SINISTRA: $IF(LAST == SUSPENDED \rightarrow LEFT)$
- SE CI TROVIAMO IN UNO SCENARIO NUMERICO LA BUSSOLA SALVA L'ULTIMO NUMERO PROCESSATO, CONTROLLANDO SE STIAMO TORNANDO DALLA CHIAMATA $n-1$:
 $IF(LAST == SUSPENDED \rightarrow n-1)$
- SE LAVORIAMO CON ARRAY/INDICI LA BUSSOLA SALVA L'ULTIMO INDICE PROCESSATO, LAVORA SU STACK MULTIPLI PER GLI INDICI (START, MID, END) E CONTROLLANDO SE L'INDICE SALVATO CORRISPONDE ALL'INDICE IN CUI ABBIAMO "TAGLIATO" LA PRIMA PORZIONE DI ARRAY: $IF(LAST == PEEKSPLITINDEX)$

3. GESTIONE DEL CASO BASE (COSA SUCCEDDE IN FONDO)

INDIVIDUIAMO IL CASO BASE DELL'ALGORITMO (ARRAY DI 1 ELEMENTO, NODO NIL...). IN QUEL PUNTO AVREMO 3 AZIONI FISSE

1. CALCOLARE L'EVENTUALE VALORE DI RITORNO NEUTRO (0, -1, 1, ...)
2. AGGIORNARE LA BUSSOLA ($LAST = CURR$)
3. INNESCARE LA SALITA ($ISDESC = FALSE$)

4. MAPPATURA AREE DI ESECUZIONE

DIVIDIAMO LE ISTRUZIONI DEL CODICE IN 3 BLOCCHI SEMANTICI:

- [AREA 1: PRE-CHIAMATA]: TUTTO CIÒ CHE AVVIENE PRIMA DELLA CHIAMATA, VA INSERITO NELLA FASE DI DISCESA DEL CICLO ($ISDESC == TRUE$)
- [AREA 2: INTER-CHIAMATA]: TUTTO CIÒ CHE AVVIENE TRA PRIMA E SECONDA CHIAMATA, VA INSERITO ALL'INTERNO DELL'IF DELLA FASE DI RISALITA
- [AREA 3: POST-CHIAMATA]: TUTTO CIÒ CHE AVVIENE DOPO L'ULTIMA CHIAMATA RICORSIVA, VA INSERITO NEL BLOCCO ELSE FINALE DELLA RISALITA

UNIVERSAL-ITERATIVE-PATTERN(PARAM)

CURR = PARAM

ISDESC = TRUE

LAST = NIL // OPPURE -1

RESULT = VALORE_NEUTRO

CONTEXTSTACK = NIL

RESULTSTACK = NIL // SOLO SE LA FUNZIONE HA VALORE DI RITORNO

WHILE (ISDESC || CONTEXTSTACK != NIL)

IF (ISDESC)

IF (CURR != CASO_BASE)

ESEGUIRE LE OPERAZIONI DI [AREA 1]

CONTEXTSTACK = PUSH (CONTEXTSTACK, CURR)

CURR = CURR → LEFT // 0 CURR - 1

ELSE // GESTIONE CASO BASE

RESULT = VALORE_CASO_BASE

LAST = CURR

ISDESC = FALSE

ELSE // FASE DI RISALITA

SUSPENDED = PEEK (CONTEXTSTACK)

IF (LAST == SUSPENDED → LEFT) // 0 (LAST == SUSPENDED - 1)

ESEGUIRE LE OPERAZIONI DI [AREA 2]

RESULTSTACK = PUSH (RESULTSTACK, RESULT) // SE C'È VALORE DI RITORNO

CURR = SUSPENDED → RIGHT // 0 (SUSPENDED - 2)

ISDESC = TRUE

ELSE

PASTRESULT = POP (RESULTSTACK) // SE NECESSARIO

ESEGUIRE LE OPERAZIONI DI [AREA 3]

POP (CONTEXTSTACK)

LAST = SUSPENDED

L'ECCEZIONE: LA RICORSIONE IN CODA

È BENE SOTTOLINEARE CHE IL PATTERN UNIVERSALE E GLI STACK DI SOSTEGNO SONO NECESSARI SOLO QUANDO LA CHIAMATA RICORSIVA CI SONO ANCORA ISTRUZIONI DA ESEGUIRE. NEL CASO IN CUI LA CHIAMATA RICORSIVA FOSSE L'ULTIMA ISTRUZIONE DELL'ALGORITMO, L'ARCHITETTURA SI SEMPLIFICA DRASTICAMENTE, POICHÉ NON CI SONO OPERAZIONI IN SOSPESO AL TERMINE DELLA CHIAMATA, NON È NECESSARIO SALVARE IL CONTESTO IN MEMORIA.

TUTTO SI RIDUCE AD UN CICLO WHILE IN CUI I PARAMETRI VENGONO AGGIORNATI SUL POSTO

ESEMPIO: RICERCA IN UN ABR

SEARCH-ABR-IT(T, k)

CURRENTPTR = T

WHILE (CURRENTPTR $\neq$ NIL && CURRENTPTR.KEY $\neq$ KEY)

IF (CURRENTPTR.KEY $> k$)

CURRENTPTR = CURRENTPTR $\rightarrow$ LEFT

ELSE

CURRENTPTR = CURRENTPTR $\rightarrow$ RIGHT

RETURN CURRENTPTR